

Programming Coursework Summary

Disa Degerholm

July 4, 2026

1 Code Style & Layout

1.1 Indentation & Spacing

- **Indentation:** Always use exactly 4 spaces per indentation level without mixing tabs.

Example: If statements must indent their nested logic block by exactly four empty spaces.

- **Operators:** Place a single space on both sides of assignment, mathematical, and comparison operators.

Example: Write `x = y + 2` instead of `x=y+2`.

- **Brackets:** Do not leave spaces immediately inside parentheses or brackets, nor between function names and brackets.

Example: Write `print("test")` and `items[0]` tightly without internal padding spaces.

1.2 Blank Lines

- **Two Blank Lines:** Use exactly two empty rows to separate top-level functions and classes.

Example: Place two blank lines down before declaring a new standalone `def global_func():`.

- **One Blank Line:** Use a single empty row to separate methods inside a class or distinct logic blocks inside a function body.

Example: Insert one blank line between the definition loops inside a calculation function block.

—

2 Definitions & Syntax Logic

2.1 Environment Setup

Intuition: Pass the active text configuration to the interpreter by pressing F5 or the green triangle.

```
1 print("Welcome to DA2005")
```

2.2 Variables & Assignment Logic

Intuition: Variables act as named boxes where evaluation flows right-to-left.

```
1 distance = 67
2 time_used = 1.25
3 average_velocity = distance / time_used
```

2.3 Print Separators

Intuition: Commas inside execution brackets natively inject a space separation.

```
1 print("Velocity:", average_velocity)
```

2.4 Multi-Variable Assignment

Intuition: Commas separate parallel variable groups to match sequence layouts linearly.

```
1 x, y = 2, 3
```

2.5 Division Systems

Intuition: Single slash outputs a decimal float, double slash truncates to an integer.

```
1 z = 22 / 7
2 q = 22 // 7
```

2.6 Exponentiation Operator

Intuition: Double up the asterisk multiplication symbol to signify power scaling.

```
1 power = 2 ** 3
```

2.7 Modulo Operation

Intuition: The percentage symbol returns only the leftover remainder from a division.

```
1 remainder = 2 % 3
```

2.8 Alternating Quotes

Intuition: Swap double and single enclosing symbols to print text quotes cleanly.

```
1 quote_1 = "Sa du 'hej'?"
```

2.9 String Escaping

Intuition: Embedding a backslash before an n pushes printing down to a newline.

```
1 multiline_print = "Hipp\nnhipp\nnhurra!"
```

2.10 User Interactivity

Intuition: Halts operations, displays the prompt text, and returns a string object.

```
1 namn = input("Vad heter du?\n")
```

2.11 Robust String Searching

Intuition: Map a letter to a number safely using find; missing values yield -1.

```
1 def hexa_char_to_int(symbol: str) -> int:
2     hexa_chars = "0123456789ABCDEF"
3     return hexa_chars.find(symbol.upper())
```

2.12 Native Brackets Lookup

Intuition: Map a number to a letter by passing integer indices into square brackets.

```
1 def int_to_hexa_char(tal: int) -> str:
2     hexa_chars = "0123456789ABCDEF"
3     return hexa_chars[tal]
```

2.13 Guard Clauses

Intuition: Escape validation loops early with condition gates to prevent crashes.

```
1 def deci_to_hex(deci: int) -> str:
2     if deci == 0:
3         return "0"
```

3 Good Practice Ideas

- **English Preference:** Standardize naming, comments, and project documentation in English.
Example: Name an age variable `user_age` instead of using Swedish terms.
- **Self-Documenting Code:** Assign meaningful identities to identifiers so logic states remain obvious.
Example: Use `average_velocity` rather than the ambiguous character `v`.
- **Inline Comments:** Leverage hash marks to declare why specific strategies are applied.
Example: Write `# Safely handles mathematical limits` above boundary conditions.
- **Literal Management:** Isolate hardcoded constants at the script top for fast adjustments.
Example: Declare `TAX_RATE = 0.25` globally instead of inserting `0.25` within expressions.
- **Type Constraints:** Explicitly convert input outcomes since they are captured as text strings.
Example: Cast numerical input using `age = int(input("Age: "))`.
- **Reserved Word Auditing:** Avoid reassigning internal keywords or core tool tags.
Example: Never declare a tracking variable with native system names like `if = 4`.
- **Guard Clauses:** Deal with anomalous inputs instantly using standalone control blocks.
Example: Return zero instantly if checking collections containing zero entities.
- **Boolean Tests:** Evaluate state queries directly through native logical conditions.
Example: Validate flags with syntax structured as `if not ready:`.
- **String Searching:** Swap string index calls with find methods to suppress runtime value errors.
Example: Execute `"ABC".find("Z")` to gather a manageable negative integer flags.
- **Typo Prevention:** Maintain method verification practices to ensure file streams release memory.
Example: Ensure text handles clear active locks explicitly via the `.close()` routine.
- **Interactive Safeguards:** Loop defensive error blocks to secure input pipelines.
Example: Run user queries inside continuous checks protected by try and except blocks.

4 Project Documentation & Git

4.1 Markdown Layout

- **Structure:** Define explicit primary headers above descriptive overview paragraphs.

Example: Write `# Project Name` at the absolute first line of text layout files.

- **Compliance Checklists:** Record architectural limitations using bullet points to demonstrate obedience.

Example: Include a checklist item stating that built-in libraries were avoided.

- **Code Presentation:** Bind technical scripts inside specialized language blocks using triple backticks.

Example: Wrap programming examples within block regions declared as ````python`.

4.2 Git Version Control

- **Repository Functionality:** Think of repositories as secure project historical records tracking file adjustments.

Example: Use repository history logs to roll back accidental formatting changes.

- **Public Visibility:** Deselect visibility restrictions during cloud uploads to provide structural links.

Example: Toggle cloud visibility off from private mode inside your local desktop sync application.

4.3 Comparison Matrix

Feature	Normal Desktop Folder	GitHub Repository
Storage	Lives locally on hard drives.	Stored locally and backed up online.
Version History	Overwritten edits are lost.	Tracks every historical snapshot taken.
Sharing	Requires file compression and emailing.	Shared instantly via a public web URL.

Table 1: Desktop Folder vs. GitHub Repository